Plantilla de Diapositivas Pu++

Por: Autor de la Presentación

Club de Programación Competitiva Pu++

16 de febrero de 2026

Contenido

1 Guía de Uso del Template
- Introducción
- Entornos Personalizados
- Código Fuente
- Matemáticas
- Problemas de Práctica

Guía de Uso del Template

Esta sección unifica la documentación de funcionalidades y la estructura sugerida para nuevos temas.

Introducción

Este documento sirve como plantilla para **diapositivas de algoritmia competitiva (Pu++)**. El objetivo es mantener un formato consistente, limpio y fácil de leer para el material educativo.

Para crear una nueva sección, se recomienda:

- Copiar la estructura de esta guía o usar los archivos base como referencia.
- Dividir el contenido en subsecciones lógicas:
  - **Teoría**
  - **Implementación**
  - **Problemas**

Entornos Personalizados

El template incluye varios entornos predefinidos para estructurar las diapositivas de forma clara y visual.

Definiciones y Teoremas

Definición Formal

Utiliza este entorno para introducir formalmente nuevos conceptos, estructuras de datos o términos clave. El argumento opcional `title` permite dar un nombre descriptivo a la definición.

Propiedad Matemática

El entorno `TheoremP` sirve para destacar propiedades matemáticas fundamentales, lemas o teoremas que justifican la corrección o complejidad de un algoritmo.

Consejos y Advertencias

Consejo

Este entorno se usa para dar consejos o recomendaciones prácticas que no son necesariamente reglas estrictas, pero ayudan a resolver problemas de manera más eficiente.

Cuidado

Úsalo para advertir sobre errores comunes, desbordamientos de enteros (overflow), casos borde complicados o complejidad temporal excesiva.

Ejemplos

Caso de Prueba

Este cuadro es ideal para mostrar ejemplos concretos, como casos de entrada y salida, trazas de ejecución paso a paso, o diagramas explicativos.

Input:

```
5
1 2 3 4 5
```

Output:

```
15
```

Código Fuente

El código se resalta automáticamente usando `minted`.

Buenas Prácticas

Consejo

Recomendación: Se recomienda mantener el código en archivos `.cpp` separados dentro de la carpeta `code/` y usar `\twocolumnminted` para incluirlos. Esto asegura que el código sea compilable y testeable independientemente del documento.

Ejemplo: Archivo externo

Ejemplo incluyendo el archivo `code/template.cpp` directamente:

```cpp
/**
 * Estructura estándar para Competitive Programming
 * Incluye optimizaciones de I/O y definiciones de tipos comunes.
 */
#include <bits/stdc++.h>
using namespace std;

// --- Tipos de datos comunes ---
using ll = long long;
using vi = vector<int>;
using pii = pair<int, int>;

// --- Constantes ---
const int INF = 1e9;
const ll LINF = 1e18;
const int MOD = 1e9 + 7;

// --- Solución del problema ---
void solve() {
    int n;
    cin >> n;

    // Tu lógica aquí
    cout << "Input: " << n << "\n";
}

int main() {
    // Optimización para I/O rápido
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int t = 1;
    // cin >> t; // Descomentar para problemas con múltiples casos de prueba
    while(t--) {
        solve();
    }

    return 0;
}
// Force cache refresh
```

Ejemplo: Código inline

También es posible escribir código directamente en el archivo `.tex` usando el entorno `minted`, útil para snippets cortos:

```
// Ejemplo de código inline
void solve() {
    int n; cin >> n;
    cout << n * 2 << endl;
}
```

Matemáticas en la Plantilla

Podemos usar comandos personalizados definidos en `macros.tex` para simplificar la notación común en algoritmia:

- **Complejidad:** $\mathcal{O}(N \log N)$, $\tilde{\mathcal{O}}(N)$, $\Omega(N)$
- **Conjuntos Numéricos:** $\mathbb{R}$ (Reales), $\mathbb{Z}$ (Enteros), $\mathbb{N}$ (Naturales)
- **Combinatoria:** $\binom{n}{k}$ para coeficientes binomiales
- **Teoría de Números:** $\gcd(a, b)$, $\operatorname{lcm}(a, b)$, $a \mid b$
- **Otros:** $\lfloor x \rfloor$, $\lceil x \rceil$

Problemas de Práctica

Es fundamental incluir una lista de problemas para reforzar el tema. El comando `\practiceproblem` facilita este formato estandarizado.

Argumentos: `{Nombre}{Dificultad}{Juez}{Link}`

Lista de Problemas

- **Sum of Two Values**

  Judge: CSES

- **Maximum Subarray Sum**

  Judge: CSES

- **Counting Tilings**

  Judge: CSES

Easy

Medium

Hard

Lista de Problemas

- **Hamiltonian Flights**

  Judge: CSES

  Very Hard

- **Problema Personalizado**
  Judge: Local

  Unknown

  Link no disponible